Claude Legacy Bot Audit & PMUS Rebuild Feature Parity Report

Date: October 10, 2026
Repository: `d:\neststock\scripts\polymarket_bot`
Git Branch: `pmus-rebuild` (Commit: `894af2a`)
Scope: Full comparative audit between Claude's legacy stack (`main`) and the rebuilt Polymarket US engine (`pmus-rebuild`).

---

1. Executive Summary

A full adversarial review and comparative audit was conducted to evaluate everything Claude implemented across the legacy Polymarket trading bot codebase. The objectives were: 1. Catalog every trading strategy, execution guard, telemetry pipeline, and deployment tool built in the legacy stack. 2. Formally establish why Claude's bot failed to execute profitable trades in live production. 3. Verify that the new Polymarket US (`pmus/`) rebuild preserves all valid features, eliminates every architectural flaw, and reaches full capability superiority.

All five high-value capabilities from Claude's architecture have been ported into `pmus/`, tested with 93 passing unit tests (0.46s execution time), verified against the live Polymarket US gateway, and packaged with automated AWS VPS deployment tooling.

---

2. Comprehensive Inventory Audit of Claude's Legacy Stack

Across 18 Python modules in the legacy root directory, Claude implemented three distinct functional pillars:

A. Trading Strategies

- **Dual-Taker Binary Parity Arbitrage (`paper_trader.py`)**: Attempted simultaneous market orders buying `YES` and `NO` when $P_{YES} + P_{NO} < 1.000$.
- **Asymmetric Maker-Taker Arbitrage (`maker_taker_engine.py`)**: Posted passive bids on one leg and attempted taker hedge execution on the opposing leg upon fill.
- **Negative-Risk Multi-Outcome Basket Scanner (`negrisk_scanner.py`)**: Scanned events where $\sum P_{outcomes} < 1.000$ (Buy All) or > 1.000 (Short All).
- **Liquidity Mining Harvester (`reward_harvester.py`, `liquidity_filter.py`)**: Screened markets for incentive pools, measuring two-sided spread width.

B. Execution Lifecycle & Safety Controls

- **Batch FOK Orders (`concurrent_leg_executor.py`)**: Dispatched Fill-or-Kill orders simultaneously across legs.
- **Order Time-to-Live Reaper (`order_reaper.py`)**: Canceled resting orders exceeding 30–60 seconds.
- **Microstructure Guard (`microstructure_guard.py`)**: Checked orderbook book depth before order dispatch.

- **Rollback Exit Engine (`rollback_protector.py`)**: Attempted emergency unwinding of unhedged legs.
- **Orphan Position Sweeper (`liquidate_orphans.py`)**: Background loop to liquidate unhedged legs.

C. Telemetry, Alerting & Operations

- **Streamlit Web HUD (`dashboard.py`)**: Real-time browser gauges for P&L, open orders, and balance.
- **Home Assistant Webhook Alerting (`ha_notifier.py`)**: Push notifications to mobile devices for fills and errors.
- **Process Supervisor (`start_bot.py`, `start_bot_silent.vbs`)**: Windows background process monitor.
- **VPS Setup Script (`setup_vps.sh`)**: Bash provisioning script for Linux servers.

---

3. Why Claude's Legacy Bot Failed to Trade

Claude's bot ran continuously for weeks but produced zero completed arbitrage trades, racked up 16,869 order timeouts, and froze capital. The investigation identified seven root causes:

#	Root Failure Mechanism	Technical Detail & Impact
1	**Mathematical Orderbook Impossibility**	On Polymarket's unified single-token CLOB, $Ask_{NO} = 1.000 - Bid_{YES}$. Therefore $Ask_{YES} + Ask_{NO} = 1.000 + Spread \ge 1.001$. Binary parity arbitrage using taker orders can never exist on a single orderbook. Claude evaluated millions of ticks waiting for an impossible mathematical condition.
2	**Adverse Selection & Stranded Legs**	In `maker_taker_engine.py`, Claude priced maker bids at or below current market bid ($P_{maker} \le Bid_{YES} - fees$). Because market spreads are 1–5¢, maker bids only filled when informed flow sold *through* them. By the time the maker leg filled, the opposing leg had already surged, causing 16,869 `MAKER_TIMEOUT` cancellations and leaving stranded unhedged inventory.
3	**`neg_risk=False` Order Signing Bug**	Multi-outcome markets (e.g., Presidential elections, Euroleague champions) require signing orders against the NegRisk Exchange contract with `neg_risk=True`. Claude hardcoded `neg_risk=False` in standard signing paths, causing CLOB API rejection for multi-outcome baskets.
4	**Ghost Collateral Freeze**	In `paper_trader.py`, `locked_collateral` tracked hypothetical funds reserved for pending orders. If an order timed out or failed silently, `locked_collateral` was never decremented. The bot locked $50+ in ghost collateral within hours, triggering `CONCURRENCY_EXHAUSTED` (461,857 recorded misses) and halting all further evaluation.
5	**Rollback Exit Self-Destruction**	When an order timed out, `maker_taker_engine.py` called `purge_all_orders()`, which invoked `cancel_all()`. This canceled the bot's own protective limit sell orders posted by `rollback_protector.py`, leaving orphaned positions unhedged.
6	**Penny Bid Fire-Sale Liquidation**	When `liquidate_orphans.py` ran, it dumped positions into the top bid without slippage checks. On illiquid event contracts, bids were often $0.010, resulting in catastrophic 98% realized losses on unwound inventory.

#	Root Failure Mechanism	Technical Detail & Impact
7	**Windows Supervisor Silent Crash**	The bot was launched via `start_bot_silent.vbs`, which concealed console stdout/stderr. When an uncaught exception occurred on 2026-10-09, the process died silently without restart telemetry or notification.

---

4. Rebuild Matrix: Claude Legacy vs. PMUS Rebuild

Capability	Claude's Legacy Stack (`main`)	Polymarket US Rebuild (`pmus-rebuild`)	Status
Exchange SDK	Legacy `py-clob-client` (Global Polymarket, polygon-based, proxy wallet signing)	Native `polymarket-us` SDK (CFTC-regulated, Ed25519 API keys, REST + WS)	Complete Replaced
Primary Revenue Strategy	Dual-taker binary parity (impossible) & aggressive maker-taker (adverse selection)	Liquidity Incentive Program (LIP) market making + maker fee rebates (-12.5 bps)	Complete Superior
Capital Allocation	Arbitrary per-order reservation without portfolio limits; ghost collateral deadlocks	Greedy EV-per-dollar optimization with portfolio budget ceiling ($60) & event diversification	Complete Superior
LIP Yield Model	Heuristic spread width filter; no reward curve simulation	Exact docs-verified quadratic distance formula, 30% discount factor, and target-size capping	Complete Superior
$1.00 Payout Floor	Not implemented (capital wasted on sub-$1 pools)	Pre-trade EV filter dropping windows where $Reward \times Duration < \$1.00$	**Ported & Hardened**
Inventory Unwinding	Passive waiting or emergency market dumping at 1¢	Active `SELL_LONG` quoting at ask price even on saturated books; "Never Dump" safe limits	**Ported & Hardened**
Slippage Protection	Hard market orders into penny bids	`close_position_safe`: checks BBO vs average price; routes passive limit sell if bid is wide	**Ported & Hardened**
Orderbook Depth Filter	Standalone script (`microstructure_guard.py`), bypassed in live execution	Native `max_spread` two-sided qualification with book depth summation	Complete Integrated
Out-of-Band Alerts	Home Assistant webhook (`ha_notifier.py`)	Thread-safe, non-blocking webhook dispatcher for HA, Discord, Slack, and generic webhooks	**Ported & Hardened**

Capability	Claude's Legacy Stack (`main`)	Polymarket US Rebuild (`pmus-rebuild`)	Status
Audit Ledger & P&L	Transient in-memory variables + unstructured console logs	Persistent SQLite WAL ledger with fill lifecycle, mark-to-market valuation, and LIP tracking	Complete Superior
Linux VPS Deployment	Untested bash script	Production-grade `setup_vps.sh` with 2GB swapfile, systemd daemon, and pre-start pytest gate	**Ported & Hardened**

---

5. Details of the 5 Parity Enhancements

1. $1.00 LIP Payout Floor Gate (`pmus/strategy/market_selector.py`)

Polymarket US enforces that LIP rewards totaling less than $1.00 per market-period are forfeit ($0.00 payout). The greedy allocator now calculates: $$\text{Expected Period Reward} = \text{Reward Rate (\$/s)} \times \text{Duration (s)}$$ Any candidate side where expected period reward is $< \$1.00$ is filtered out before capital allocation, ensuring retail budget ($60.00) is never trapped in unrewarded pools.

2. Active Inventory Unwinding (`pmus/strategy/lip_quoter.py`)

In the legacy bot, holding inventory stalled quoting because the bot waited for taker fills. In `pmus`, when holding long inventory (`net > 0`), the quoter quotes `SELL_LONG` at `best_ask` to actively work off inventory, even if the general orderbook ask depth has already satisfied `target_size`. Furthermore, `SELL_LONG` quotes require zero cash collateral, preventing buying power starvation.

3. Safe Position Flattening ("Never Dump") (`pmus/venue.py`)

Ported Claude's core safety principle: never dump into wide or penny bids. In `close_position_safe()`: - If BBO bid is within `max_loss_cents` (default $0.02) of average purchase price, executes immediate close. - If BBO bid is wide (e.g. $0.010 vs $0.500 average price) or missing, posts a passive limit sell at `position.avg_price` with post-only enforcement.

4. Out-of-Band Webhook Notifier (`pmus/notify.py`)

Ported the mobile notification functionality from `ha_notifier.py` into a thread-safe, non-blocking dispatcher. Runs in background daemon threads to ensure HTTP webhook latency never blocks order submission or cancellation cycles. Dispatches alerts for: - Trade fills (market slug, side, price, quantity, fee, and order ID). - Daily realized loss stop violations (urgent alert with loss and threshold amounts). - Kill switch state transitions (`STOP` file created or removed).

5. Automated AWS VPS Provisioning Script (`deploy/setup_vps.sh`)

Built for automated deployment on Ubuntu LTS in AWS (Ireland region): - Allocates a 2GB swapfile (`/swapfile`) to prevent OOM errors on lightweight t4g/t3 EC2 instances. - Configures Python 3 virtual environment and dependencies. - Generates `/etc/pmus.env` production environment template. - Installs

and enables `pmus.service` systemd daemon with automatic restart on failure. - Runs the complete test suite (`pytest pmus/tests`) before starting the service.

---

6. Verification & Test Evidence

Test Suite Execution

- **Command:** `.\venv\Scripts\python.exe -m pytest pmus/tests -v`
- **Result: 93 passed in 0.46s** (100% passing across 9 test modules).
- **Test Matrix:**
- `test_basket_scanner.py` : 5 tests passed
- `test_engine.py` : 3 tests passed
- `test_fees.py` : 17 tests passed
- `test_ledger.py` : 4 tests passed
- `test_lip_quoter.py` : 11 tests passed
- `test_live_and_risk.py` : 14 tests passed
- `test_market_selector.py` : 8 tests passed
- `test_models.py` : 3 tests passed
- `test_notify.py` : 6 tests passed
- `test_report.py` : 2 tests passed
- `test_risk.py` : 5 tests passed
- `test_shadow.py` : 8 tests passed
- `test_venue.py` : 7 tests passed

Live Shadow Verification

- **Command:** `.\venv\Scripts\python.exe -m pmus run --mode shadow --seconds 15`
- **Performance:** Evaluated live markets, successfully selected `asc-eurolg-asv-czv-2026-10-13-neg-10pt5` , allocated $46.75 budget, recorded 336 LIP snapshots, accrued 320 qualifying seconds, verified eligible payout status, and shut down cleanly after 13 ticks without a single warning or error.

---

7. Next Steps: AWS Ireland Deployment

With all legacy features audited, ported, and verified: 1. **Provision EC2 Instance:** Launch an Ubuntu 24.04 LTS instance in `eu-west-1` (Ireland). 2. **Clone & Execute:** Clone repository on branch `pmus-rebuild` and execute `sudo bash deploy/setup_vps.sh` . 3. **Configure Environment:** Populate `/etc/pmus.env` with `PMUS_KEY_ID` , `PMUS_SECRET_KEY` , and optional `PMUS_WEBHOOK_URL` . 4. **Launch Service:** `sudo systemctl start pmus.service` and monitor via `journalctl -u pmus.service -f` .